

Reinforcement Learning

Clase 04

Ignacio Brevis

`ignaciobrve@unisabana.edu.co`

Universidad de La Sabana

09 septiembre 2026



Para lidiar con la estocasticidad de los sistemas, definimos la **función de valor** (value function o state-value function).

State-value function

Dada una política π y $s \in \mathcal{S}$. La **función de valor** se define como valor esperado de G_t

$$v_{\pi}(s) := \mathbb{E}[G_t | S_t = s]$$

- $v_{\pi}(s)$ depende de s .
- $v_{\pi}(s)$ depende de la política π .
- $v_{\pi}(s)$ no depende de t .

Obs: Cuando la política y la dinámica (transición y recompensa) son determinísticas, un estado inicial siempre generará la misma trayectoria.

Obs: Si la política y la dinámica son estocásticas, un estado inicial puede dar lugar a diferentes trayectorias. Las recompensas de diferentes trayectorias pueden ser distintas, y la función de valor será el valor esperado de dichas recompensas.

$$\begin{aligned} \rightarrow \quad \underline{v_\pi(s)} &= \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{r \in \mathcal{R}} p(r|s, a)r + \gamma \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} p(s'|s, a) \underline{v_\pi(s')}, \\ &= \sum_{a \in \mathcal{A}} \pi(a|s) \left[\sum_{r \in \mathcal{R}} p(r|s, a)r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a)v_\pi(s') \right], \quad \forall s \in \mathcal{S} \end{aligned}$$

es la **ecuación de Bellman**.

- Notemos que $v_\pi(s)$ y $v_\pi(s')$ son las incógnitas.
- Para resolver la ecuación, tenemos que generar un sistema con tantas ecuaciones como elementos hay en $\mathcal{S}$.
- $\pi := \pi(a|s)$ es una política dada, y la función de valor nos permite evaluarla. Resolver la ecuación de Bellman nos permite realizar dicha evaluación.
- $p(r|s, a)$ y $p(s'|s, a)$ nos proporcionan la dinámica del sistema.

Repaso: Bellman equation

Bellman equation

$$v_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{r \in \mathcal{R}} p(r|s, a) r + \gamma \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} p(s'|s, a) v_{\pi}(s'),$$

$$\underline{r_{\pi}(s)} := \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{r \in \mathcal{R}} p(r|s, a) r, \quad y \quad \underline{p_{\pi}(s'|s)} := \sum_{a \in \mathcal{A}} \pi(a|s) p(s'|s, a)$$

$$v_{\pi}(s_i) = r_{\pi}(s_i) + \gamma \sum_{s_j \in \mathcal{S}} p_{\pi}(s_j|s_i) v_{\pi}(s_j)$$

Forma matricial: $v_{\pi} \in \mathbb{R}^n$, $r_{\pi} \in \mathbb{R}^n$ y $P_{\pi} \in \mathbb{R}^{n \times n}$, donde $[v_{\pi}]_i = v_{\pi}(s_i)$, $[r_{\pi}]_i = r_{\pi}(s_i)$ y $[P_{\pi}]_{ij} = p_{\pi}(s_j|s_i)$ con .

$$\Rightarrow \boxed{v_{\pi} = r_{\pi} + \gamma P_{\pi} v_{\pi}}$$

Repaso: Ejemplo 1

Calcular el sistema de Bellman (tomando $\gamma = 0.9$)

$$v_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left[\sum_{r \in \mathcal{R}} p(r|s, a)r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a)v_{\pi}(s') \right]$$

$$\begin{cases} v_{\pi}(s_1) = \gamma v(s_3) \\ v_{\pi}(s_2) = 1 + \gamma v(s_4) \\ v_{\pi}(s_3) = 1 + \gamma v(s_4) \\ v_{\pi}(s_4) = 1 + \gamma v(s_4) \end{cases}$$

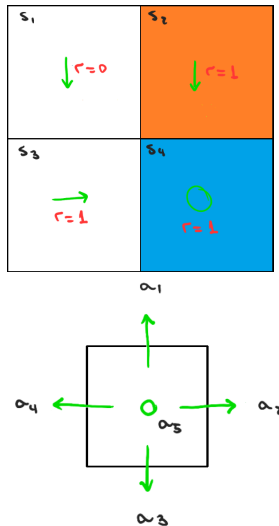


Image adapted from Zhao, Mathematical Foundations of Reinforcement Learning.

Repaso: Ejemplo 2

Calcular el sistema de Bellman (tomando $\gamma = 0.9$)

$$v_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left[\sum_{r \in \mathcal{R}} p(r|s, a)r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a)v_{\pi}(s') \right]$$

$$\begin{cases} v_{\pi}(s_1) = -0.5 + \frac{\gamma}{2}v(s_2) + \frac{\gamma}{2}v(s_3) \\ v_{\pi}(s_2) = 1 + \gamma v(s_4) \\ v_{\pi}(s_3) = 1 + \gamma v(s_4) \\ v_{\pi}(s_4) = 1 + \gamma v(s_4) \end{cases}$$

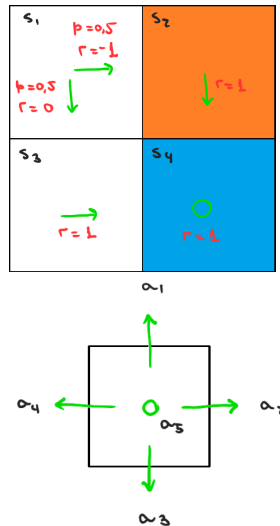


Image adapted from Zhao, Mathematical Foundations of Reinforcement Learning.

Repaso: Solución cerrada e iterativa

Como $v_\pi = r_\pi + \gamma P_\pi v_\pi$ es un sistema lineal, puede ser “fácilmente” resuelto al invertir $(I - \gamma P_\pi)$



$$v_\pi = (I - \gamma P_\pi)^{-1} r_\pi$$

En la práctica, calcular la **inversa** de una matriz de $n \times n$ puede ser muy **costoso**! Una alternativa es usar métodos iterativos. Por ejemplo, la ecuación de Bellman

$$v_\pi = r_\pi + \gamma P_\pi v_\pi$$

Se puede resolver de forma iterativa:



$$v_{k+1} = r_\pi + \gamma P_\pi v_k, \quad k = 0, 1, 2, \dots$$

$$v_{k+1} \xrightarrow[k \rightarrow \infty]{} v_\pi$$

Obs: Este método, no nos entrega la solución exacta de v_π , pero nos proporciona una aproximación que cada vez “se acerca más” a v_π .

Repaso: Ejemplo 1

Calcular la función de valor v_π ,

$$v_\pi = \begin{bmatrix} 3.5 & 3.9 & 4.3 & 4.8 & 5.3 \\ 3.1 & 3.5 & 4.8 & 5.3 & 5.9 \\ 2.8 & 2.5 & 10. & 5.9 & 6.6 \\ 2.5 & 10. & 10. & 10. & 7.3 \\ 2.3 & 9. & 10. & 9. & 8.1 \end{bmatrix}$$

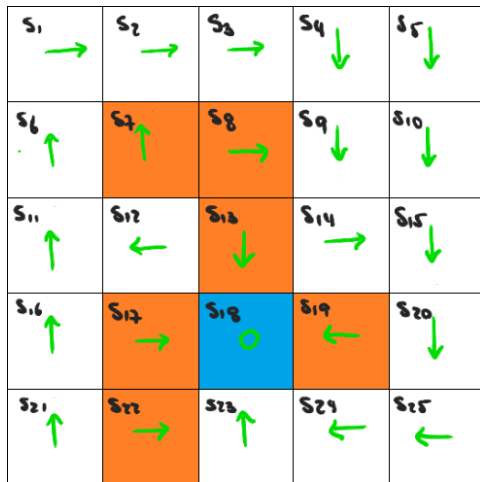


Image adapted from Zhao, Mathematical Foundations of Reinforcement Learning.

Repaso: Ejemplo 2

Calcular la función de valor v_π para el siguiente juego:

- $\mathcal{A} = \{\uparrow, \rightarrow, \downarrow, \leftarrow\}$
- En cada celda, cada una de las 4 acciones es igualmente probable ($\pi(a|s_i) = 1/4$, con $a \in \mathcal{A}$).
- Acciones que lleven al agente fuera del entorno, dejan al agente en la misma celda y generan una recompensa de $r = -1$.
- Desde s_2 , todas las acciones tienen una recompensa de +10 y llevan al agente a s_{22} .
- Desde s_4 , todas las acciones tienen una recompensa de +5 y llevan al agente a s_{14} .
- Todas las otras acciones tienen una recompensa $r = 0$.

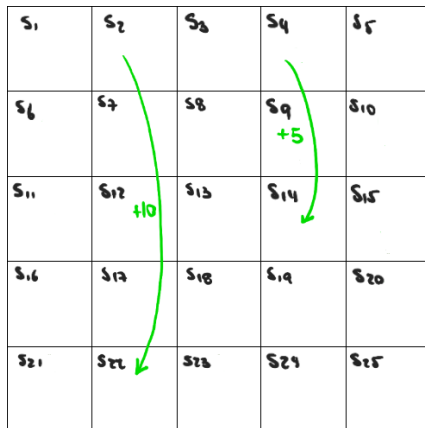


Image adapted from Sutton and Barto, Reinforcement Learning: An Introduction.

Repaso: Ejemplo 2

```
P_pi = np.array([
    [0.5, 0.25, 0., 0., 0., 0.25, 0., 0., 0., 0.],
    [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
    [0., 0.25, 0.25, 0.25, 0., 0., 0., 0., 0.25, 0.],
    [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
    [0., 0., 0., 0.25, 0.5, 0., 0., 0., 0., 0.],

    [0.25, 0., 0., 0., 0., 0., 0.25, 0.25, 0., 0.],
    [0., 0.25, 0., 0., 0., 0., 0.25, 0., 0.25, 0.],
    [0., 0., 0.25, 0., 0., 0., 0., 0.25, 0., 0.],
    [0., 0., 0., 0.25, 0., 0., 0., 0., 0.25, 0.],
    [0., 0., 0., 0., 0.25, 0., 0., 0., 0., 0.]
])
```

$\gamma_{\pi}(s)$

```
r_pi = np.array([
    -0.5, # s1
    10.0, # s2 (A)
    -0.25, # s3
    5.0, # s4 (B)
    -0.5, # s5

    -0.25, # s6
    0.0, # s7
    0.0, # s8
    0.0, # s9
    -0.25, # s10
])
```

$p(s_1|s_1) = 0.5$
 $p(s_2|s_1) = 0.25$
 $p(s_6|s_1) = 0.25$
 $p(s_j|s_1) = 0$
 $j \neq 1, 2, 6$

s ₁	s ₂	s ₃	s ₄	s ₅
s ₆	s ₇	s ₈	s ₉ +5	s ₁₀
s ₁₁	s ₁₂ +10	s ₁₃	s ₁₄	s ₁₅
s ₁₆	s ₁₇	s ₁₈	s ₁₉	s ₂₀
s ₂₁	s ₂₂	s ₂₃	s ₂₄	s ₂₅

Image adapted from Sutton and Barto, Reinforcement Learning: An Introduction.

Repaso: Ejemplo 2

Resolviendo la ecuación de Bellman

$(I - \gamma P_\pi)v_\pi = r_\pi$ para calcular v_π , obtenemos:

$$v_\pi = \begin{bmatrix} 3.3 & 8.8 & 4.4 & 5.3 & 1.5 \\ 1.5 & 3. & 2.3 & 1.9 & 0.5 \\ 0.1 & 0.7 & 0.7 & 0.4 & -0.4 \\ -1. & -0.4 & -0.4 & -0.6 & -1.2 \\ -1.9 & -1.3 & -1.2 & -1.4 & -2. \end{bmatrix}$$

s_1	s_2	s_3	s_4	s_5
s_6	s_7	s_8	s_9 +5	s_{10}
s_{11}	s_{12} +10	s_{13}	s_{14}	s_{15}
s_{16}	s_{17}	s_{18}	s_{19}	s_{20}
s_{21}	s_{22}	s_{23}	s_{24}	s_{25}

Image adapted from Sutton and Barto, Reinforcement Learning: An Introduction.

Función de valor (state-value function)

$$v_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left[\sum_{r \in \mathcal{R}} p(r|s, a)r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a)v_{\pi}(s') \right], \quad \forall s \in \mathcal{S}$$

La **función de acción-valor** (para la política π) se define como:

$$\rightarrow q_{\pi}(s, a) = \mathbb{E}[G_t | S_t = s, \underline{A_t = a}]$$

Obs: $\underline{v_{\pi}(s)} = \mathbb{E}[G_t | S_t = s] = \sum_a \underbrace{\mathbb{E}(G_t | S_t = s, A_t = a)}_{q_{\pi}(s, a)} \underbrace{p(a|s)}_{\pi(a|s)}$

$$\boxed{v_{\pi}(s) = \sum_a q_{\pi}(s, a) \pi(a|s)}$$

Relación entre estado-valor y acción-valor

$$\rightarrow v_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) q_{\pi}(s, a)$$

✱ **Obs:** La función de acción-valor, como su nombre lo dice, indica el “valor” de tomar una acción a en el estado s .

Obs: Recordando que

$$\star v_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left[\underbrace{\sum_{r \in \mathcal{R}} p(r|s, a)r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a)v_{\pi}(s')}_{q_{\pi}(s, a)} \right],$$

Notemos que

$$\rightarrow \boxed{q_{\pi}(s, a) = \sum_{r \in \mathcal{R}} p(r|s, a)r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) \underline{v_{\pi}(s')}}}$$

Action-value function: Ejemplo

$$q_{\pi}(s, a) = \sum_{r \in \mathcal{R}} p(r|s, a)r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a)v_{\pi}(s')$$

Calcular

$$\begin{aligned} q_{\pi}(s_1|a_2) &= \sum_r p(r|s_1, \rightarrow) r + \gamma \sum_{s'} p(s'|s_1, \rightarrow) v_{\pi}(s') \\ &= (1)(-1) + \gamma (1)v_{\pi}(s_2) \end{aligned}$$

$$q_{\pi}(s_1|a_2) = -1 + \gamma v_{\pi}(s_2)$$

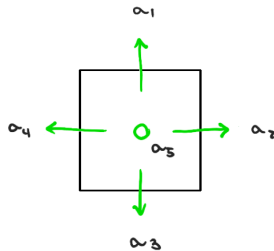
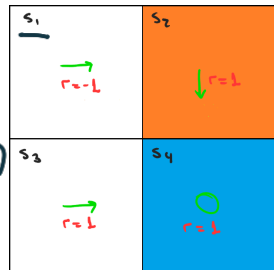


Image adapted from Zhao, Mathematical Foundations of Reinforcement Learning.

Action-value function: Ejemplo

$$q_{\pi}(s, a) = \sum_{r \in \mathcal{R}} p(r|s, a)r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a)v_{\pi}(s')$$

$$\ast q_{\pi}(s_1|a_1) = -1 + \gamma v_{\pi}(s_1)$$

$$q_{\pi}(s_1|a_3) = 0 + \gamma v_{\pi}(s_3)$$

$$q_{\pi}(s_1|a_4) = -1 + \gamma v_{\pi}(s_1)$$

$$q_{\pi}(s_1|a_5) = 0 + \gamma v_{\pi}(s_1)$$

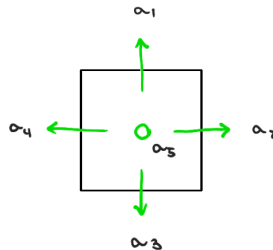
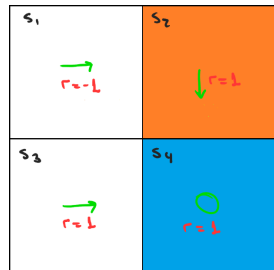


Image adapted from Zhao, Mathematical Foundations of Reinforcement Learning.

$$q_{\pi}(s, a) = \sum_{r \in \mathcal{R}} p(r|s, a)r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a)v_{\pi}(s')$$

- Dado $s \in \mathcal{S}$, podemos calcular $q_{\pi}(s, a)$ para cualquier acción a . Incluso si la política “no considera” esa acción.
- * • $q_{\pi}(s, a)$ toma la acción a y luego sigue la política π en las acciones futuras.
- * • A pesar de que algunas acciones no son consideradas por una política, no significa que no sean “buenas acciones”.
- * • Esto será especialmente útil cuando busquemos políticas óptimas.

Obs: $v_{\pi}(s_1) = \sum_a \pi(a|s_1)q_{\pi}(s_1, a) = \underbrace{\pi(\uparrow|s_1)}_{=0} q_{\pi}(s_1, \uparrow) + \underbrace{\pi(\rightarrow|s_1)}_1 q_{\pi}(s_1, \rightarrow) + \dots$
 $= q_{\pi}(s_1, \rightarrow) = -1 + \gamma v_{\pi}(s_2)$

Optimal state values y optimal policies

Recordemos que

funcion de valor

$$\star \begin{cases} v_{\pi}(s_1) = -1 + \gamma v_{\pi}(s_2) \\ v_{\pi}(s_2) = +1 + \gamma v_{\pi}(s_4) \\ v_{\pi}(s_3) = +1 + \gamma v_{\pi}(s_4) \\ v_{\pi}(s_4) = +1 + \gamma v_{\pi}(s_4) \end{cases}$$

Si tomamos $\gamma = 0.9$, tenemos que

$$v_{\pi}(s_1) = \underline{8} \text{ y } v_{\pi}(s_2) = v_{\pi}(s_3) = v_{\pi}(s_4) = \underline{10}$$

$$\begin{aligned} q_{\pi}(s_1, a_1) &= -1 + \gamma v_{\pi}(s_1) = -1 + 0.9 \cdot 8 = 6.2 \\ \rightarrow q_{\pi}(s_1, a_2) &= -1 + \gamma v_{\pi}(s_2) = 8 \\ q_{\pi}(s_1, a_3) &= 0 + \gamma v_{\pi}(s_3) = 9 \\ q_{\pi}(s_1, a_4) &= -1 + \gamma v_{\pi}(s_1) = 6.2 \\ q_{\pi}(s_1, a_5) &= 0 + \gamma v_{\pi}(s_1) = 7.2 \end{aligned}$$

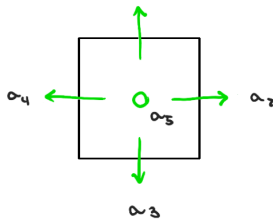
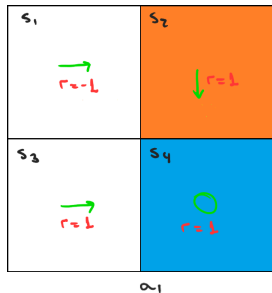


Image adapted from Zhao, Mathematical Foundations of Reinforcement Learning.

Política optimal

Una política π^* es **óptima** si su función de valor cumple que:

$$\underline{v_{\pi^*}(s)} \geq \underline{v_{\pi}(s)}, \quad \underline{\forall s \in \mathcal{S}}$$

para cualquier otra política π .

Obs: $\underline{v_{\pi^*}(s)}$ es el valor óptimo del estado s .

Nos interesa saber 3 cosas:

- Existencia? ✓
- Unicidad? ✓ (no de la política)
- Algoritmo? ✓

Recuerdo de optimización

Consideremos el siguiente problema de optimización. Cómo resolvemos el siguiente problema:

$$x = \max_y (2x - 1 - y^2)$$

$$\max_y (2x - 1 - y^2)$$

① resolvemos para y

$$\max_y (2x - 1 - y^2) = 2x - 1 \quad (\text{cuando } y=0)$$

② resolvemos para x

$$\begin{array}{l|l} x = \max_y (2x - 1 - y^2) & x = 2x - 1 \\ = 2x - 1 & 1 = x \\ & \boxed{x = 1} \end{array}$$

Ecuación de optimalidad de Bellman (BOE)

$$v(s) = \max_{\pi(s)} \sum_{a \in \mathcal{A}} \pi(a|s) q(s, a),$$

donde

$$q(s, a) = \sum_{r \in \mathcal{R}} p(r|s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v(s')$$

- La BOE tiene dos incógnitas, la función de valor $v(s)$ y la política π
- La BOE “resuelve la ecuación de Bellman”, pero en vez de tomar una política dada, busca la política que maximiza $v(s)$
- La BOE nos permite encontrar políticas óptimas y valores de estados óptimos

Bellman optimality equation

Consideremos un entorno con n estados $\{s_1, s_2, \dots, s_n\}$, entonces la ecuación optimal de Bellman se puede escribir de la forma:

$$\begin{bmatrix} v(s_1) \\ v(s_2) \\ \vdots \\ v(s_n) \end{bmatrix} = \max_{\pi \in \Pi} \left(\begin{bmatrix} r_\pi(s_1) \\ r_\pi(s_2) \\ \vdots \\ r_\pi(s_n) \end{bmatrix} + \gamma \begin{bmatrix} p_\pi(s_1|s_1) & p_\pi(s_2|s_1) & \cdots & p_\pi(s_n|s_1) \\ p_\pi(s_1|s_2) & p_\pi(s_2|s_2) & \cdots & p_\pi(s_n|s_2) \\ \vdots & \vdots & \ddots & \vdots \\ p_\pi(s_1|s_n) & p_\pi(s_2|s_n) & \cdots & p_\pi(s_n|s_n) \end{bmatrix} \begin{bmatrix} v(s_1) \\ v(s_2) \\ \vdots \\ v(s_n) \end{bmatrix} \right),$$

$$\text{donde } r_\pi(s) := \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{r \in \mathcal{R}} p(r|s, a)r \quad \text{y} \quad p_\pi(s'|s) := \sum_{a \in \mathcal{A}} \pi(a|s)p(s'|s, a)$$

$$v = r_\pi + \gamma P_\pi v$$

Forma matricial de BOE

$$\rightarrow v = \max_{\pi \in \Pi} (r_\pi + \gamma P_\pi v)$$

$$\pi \in \Pi$$

Recuerdo de optimización (Importante!)

Definición (Contracción o función contractiva)

Se dice que $f : \mathbb{R}^d \rightarrow \mathbb{R}^d$ es una función contractiva si existe $\eta \in (0, 1)$, tal que:



$$\|f(x_1) - f(x_2)\| \leq \eta \|x_1 - x_2\|,$$

Definición (Punto fijo)

Se dice que $f : \mathbb{R}^d \rightarrow \mathbb{R}^d$ tiene un punto fijo, si existe x^* , tal que:

$$f(x^*) = x^*.$$



Teorema (Función contractiva)

Si f es una función **contractiva**, entonces:

- **Existe** un **único** punto fijo x^* .
- El punto fijo puede **calcularse iterativamente** como $x_{k+1} = f(x_k)$ (para cualquier punto inicial x_0) y $x_k \rightarrow x^*$ cuando $k \rightarrow \infty$.

Recuerdo de optimización: Ejemplo

Sea $f(x) = 1 + \frac{1}{4} \sin(x)$.

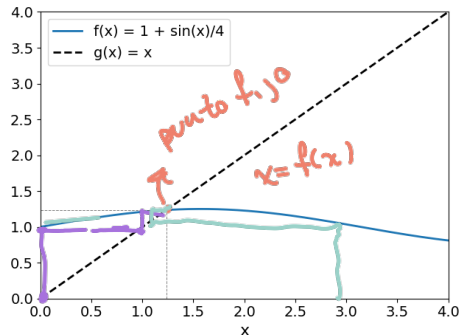
$$\eta = \frac{1}{4} \in (0,1)$$

Notemos que $|f(x) - f(y)| = \left| \frac{\sin(x)}{4} - \frac{\sin(y)}{4} \right| \leq \frac{1}{4} |x - y|$

f es contractiva!

Para resolver $x = f(x)$, utilizaremos el teorema anterior ($x_{k+1} = f(x_k)$). Partimos de un punto cualquiera, por ejemplo, $x_0 = 0$.

<u>$x_0 = 0.0$</u>	$\rightarrow$	<u>$f(x_0) = 1.0$</u>
<u>$x_1 = 1.0$</u>	$\rightarrow$	<u>$f(x_1) = 1.2104$</u>
<u>$x_2 = 1.2104$</u>	$\rightarrow$	<u>$f(x_2) = 1.2339$</u>
<u>$x_3 = 1.2339$</u>	$\rightarrow$	<u>$f(x_3) = 1.2359$</u>
$x_4 = 1.2359$	$\rightarrow$	$f(x_4) = 1.2361$
$x_5 = 1.2361$	$\rightarrow$	$f(x_5) = 1.2361$
$x_6 = 1.2361$	$\rightarrow$	<u>$f(x_6) = 1.2361$</u>



Bellman optimality equation

Resolución de la BOE

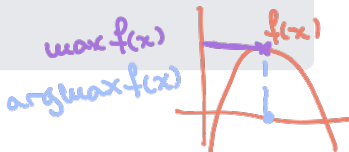
Resolver la BOE implica encontrar $\underline{v}^*$ y la política óptima $\underline{\pi}^*$ tal que:

$$v^* = f(v^*)$$

Contractivo

$$v^* = \max_{\pi \in \Pi} (r_{\pi} + \gamma P_{\pi} v^*)$$

$f(v^*)$



* Obs: $f(v) := \max_{\pi \in \Pi} (r_{\pi} + \gamma P_{\pi} v)$ es una función contractiva.

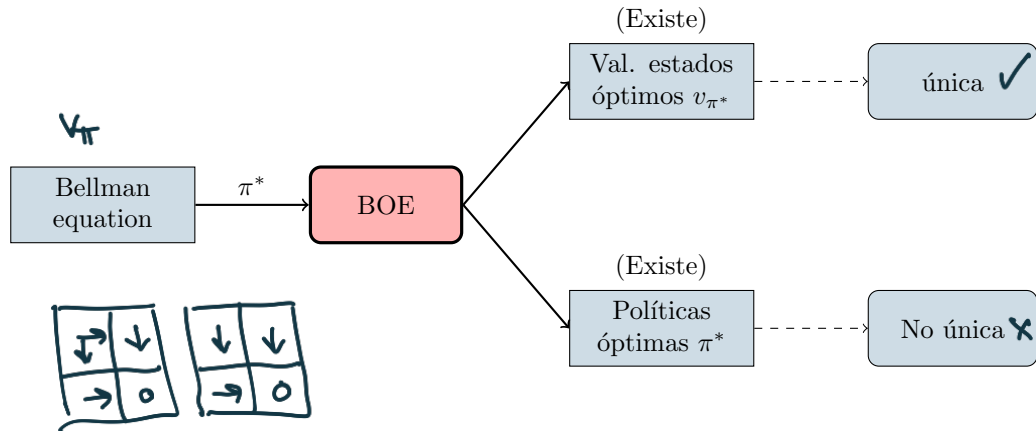
La ecuación optimal de Bellman se puede resolver usando el teorema de la función contractiva!

Esto quiere decir que siempre existe un único v^* que resuelve la BOE. Además v^* se puede calcular iterativamente de la forma

$$v_{k+1} = f(v_k) = \max_{\pi \in \Pi} (r_{\pi} + \gamma P_{\pi} \underline{v_k}),$$

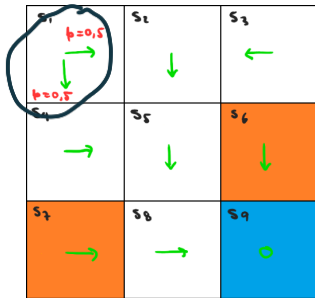
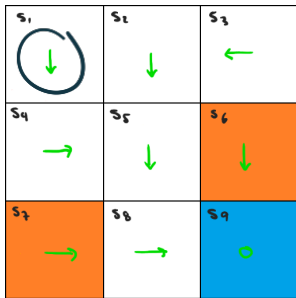
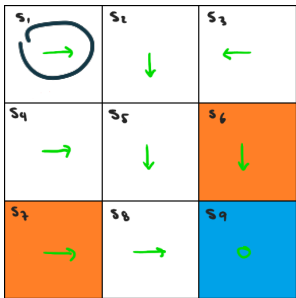
donde $v_k \rightarrow v^*$ ($k \rightarrow \infty$). Además, $\pi^* := \arg \max_{\pi \in \Pi} (r_{\pi} + \gamma P_{\pi} v^*)$ (puede no ser único).

Bellman optimality equation



Obs: Siempre existe una política óptima **determinista** (siguiendo un enfoque greedy)

Optimal policies

Tomando $\gamma = 0.9$

$$\rightarrow v_\pi = \begin{bmatrix} 7.29 & 8.10 & 7.29 \\ 8.10 & 9.00 & 10.00 \\ 9.00 & 10.00 & 10.00 \end{bmatrix}$$

Tarea

Ejercicio: Calcular la función de valor para estas tres políticas (pueden usar el código `Example_01_value_function.ipynb` en GitHub)

Policy evaluation

La **evaluación de una política** π , como su nombre lo indica, significa calcular la función de valor v_π para dicha política.

Dado un valor inicial para v_0 (normalmente $v_0(s) = 0$ para todo s),

$$\rightarrow v_{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \underbrace{\left[\sum_{r \in \mathcal{R}} p(r|s, a)r + \sum_{s' \in \mathcal{S}} p(s'|s, a)v_k(s') \right]}_{q_k(s, a)} \quad \text{un paso en } k$$

$\rightarrow$ (o de forma vectorial $v_{k+1} = (r_\pi + \gamma P_\pi v_k)$).

A medida que k aumenta, generamos una sucesión ($v_1, v_2, v_3, \dots, v_k, \dots$) que converge a la función de valor v_π .

$$\boxed{v_k(s) \rightarrow v_\pi(s)}$$

Obs: Ya hicimos esto en la parte final de `Example_01_value_function.ipynb` ✖

Policy iteration

El método de **iteración de política** nos permite calcular **políticas óptimas** usando *policy evaluation* y un **paso de mejora**.

Dada una política inicial π_0 (normalmente aleatoria), calculamos v_{π_0} usando la evaluación de política

$$v_{k+1}(s) = \sum_{a \in \mathcal{A}} \pi_0(a|s) \left[\sum_{r \in \mathcal{R}} p(r|s, a)r + \sum_{s' \in \mathcal{S}} p(s'|s, a)v_k(s') \right] \quad \text{con } k = 1, 2, 3, \dots$$

Una vez que tenemos v_{π_0} , podemos mejorar la política por medio de

$$\pi_1(s) \leftarrow \arg \max_a \underbrace{\left[\sum_{r \in \mathcal{R}} p(r|s, a)r + \sum_{s' \in \mathcal{S}} p(s'|s, a)v_{\pi_0}(s') \right]}_{q_{v_0}(s, a)}$$

(y luego iteramos sobre la política...)

Uno de los problemas de **policy iteration** es que en cada iteración de la política, requiere una evaluación de política, que es un proceso iterativo en si mismo.

El método de **iteración de valor** nos permite calcular **políticas óptimas** directamente resolviendo la **ecuación optimal de Bellman**.

$$\rightarrow v_{k+1}(s) = \max_{a \in \mathcal{A}} \underbrace{\left[\sum_{r \in \mathcal{R}} p(r|s, a)r + \sum_{s' \in \mathcal{S}} p(s'|s, a)v_k(s') \right]}_{q_k(s, a)} \quad \text{con } k = 1, 2, 3, \dots$$

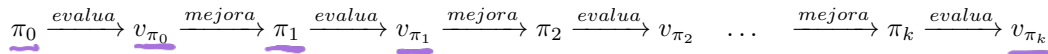
Una vez que tenemos la función de valor óptima v^* , podemos obtener (una) política óptima de la forma:

greedy approach

$$\rightarrow \pi^*(a|s) = \begin{cases} 1, & a = a^*(s), \\ 0, & a \neq a^*(s), \end{cases} \quad \text{con } \boxed{a^*(s) \leftarrow \arg \max_a q^*(a, s)} \quad [\text{puede ser no único}]$$

Policy iteration vs value iteration

Policy iteration

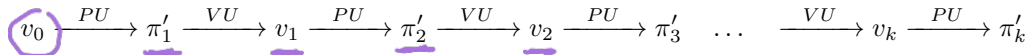


evalua: evaluación de política

mejora: mejora de política

cuando la política es estable, por ejemplo $\pi_k = \pi_{k-1}$, paramos el algoritmo y $\pi^* \approx \pi_k$.

Value iteration



PU: actualización de política (policy update)

VU: actualización de valor (value update)

cuando la función de valor es estable, por ejemplo $\|v_k - v_{k-1}\|$ es pequeño, paramos el algoritmo y $\pi^* \approx \pi'_k$.

Policy iteration vs value iteration

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow$ true

For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ false

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Policy iteration vs value iteration

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

```
|  $\Delta \leftarrow 0$   
| Loop for each  $s \in \mathcal{S}$ :  
|    $v \leftarrow V(s)$   
|    $V(s) \leftarrow \max_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$   
|    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$   
until  $\Delta < \theta$ 
```

Output a deterministic policy, $\pi \approx \pi_*$, such that

$$\pi(s) = \arg\max_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$$

Policy iteration vs value iteration

Tarea: Encontrar una política óptima para el entorno gridworld de 3×3 (siguiendo las reglas vistas en la primera clase) por medio de los métodos:

- iteración de política
- iteración de valor

(tomar $\gamma = 0.9$)

s_1	s_2	s_3
s_4	s_5	s_6
s_7	s_8	s_9